Natural Language Processing Mini-Project

Code:

train.py

```
import numpy as np
import evaluate
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding
)

# 1. Load the IMDb dataset (Sentiment Analysis: Positive vs Negative)
dataset = load_dataset("imdb")

# 2. Load the pre-trained DistilBERT tokenizer
checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

# 3. Preprocessing function to tokenize text
def tokenize_function(examples):
    return tokenizer(examples["text"], truncation=True)

# Apply tokenization to the dataset
# We select a small subset (1,000 samples) to ensure it runs quickly
tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
train_dataset = tokenized_datasets["train"].shuffle(seed=42).select(range(1000))
test_dataset = tokenized_datasets["test"].shuffle(seed=42).select(range(500))

# 4. Define evaluation metric (Accuracy)
metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)

# 5. Load the model with a classification head (2 labels: Pos/Neg)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)

# 6. Set up Training Arguments
training_args = TrainingArguments(
    output_dir="test_trainer",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_steps=10,
)

# 7. Initialize the Trainer
trainer = Trainer(
```

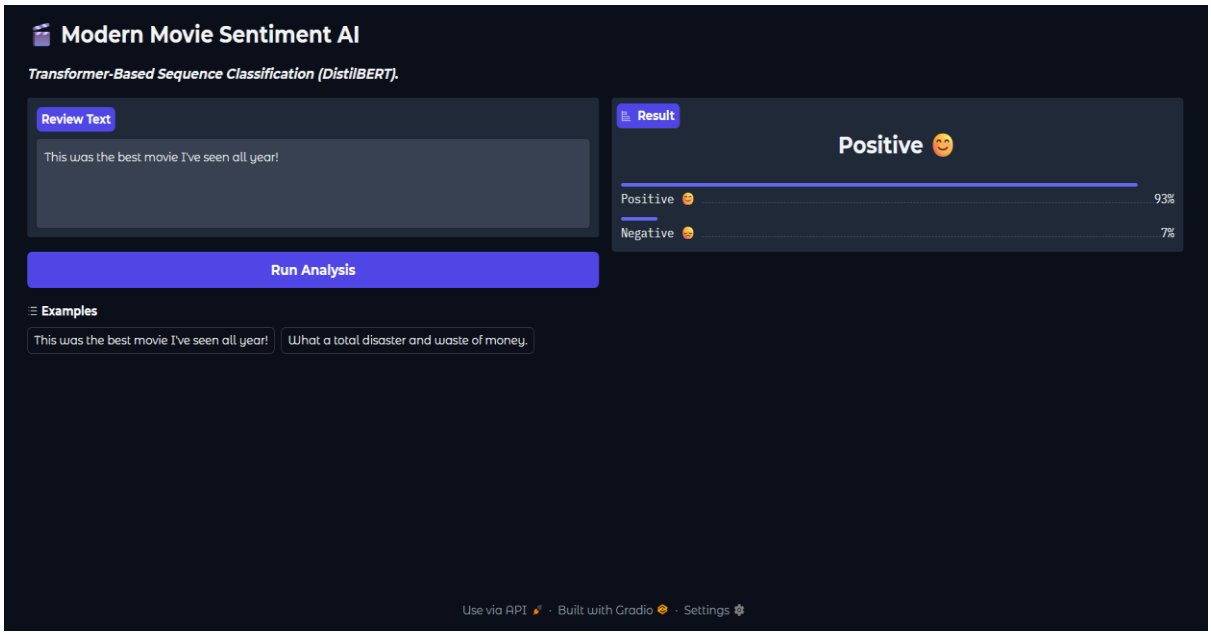
```
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    processing_class=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer=tokenizer),
    compute_metrics=compute_metrics,
)

# 8. Start Fine-tuning
print("Starting training...")
trainer.train()

# 9. Save the fine-tuned model
trainer.save_model("./my_fine_tuned_model")
print("Training complete. Model saved to ./my_fine_tuned_model")

# Inference Test
from transformers import pipeline
classifier = pipeline("sentiment-analysis", model="./my_fine_tuned_model")
print(classifier("This was an absolutely brilliant cinematic masterpiece!"))
print(classifier("This was an absolutely horrible cinematic experience!"))
```

test.py

```
import gradio as gr
from transformers import pipeline

# 1. Load the model
try:
    model_path = "./my_fine_tuned_model"
    # top_k=None tells the model to give us probabilities for BOTH labels
    classifier = pipeline("text-classification", model=model_path, top_k=None)
except Exception as e:
    print(f"Error loading model: {e}")

# 2. Refined Prediction Logic
def analyze_review(text):
    if not text or len(text.strip()) == 0:
        return {"Please enter text": 0.0}

    # Get predictions
    results = classifier(text)

    formatted_results = {}
    for item in results[0]: # Access the first element of the list
        label_name = "Positive" if item['label'] == 'LABEL_1' else "Negative"
        formatted_results[label_name] = item['score']

    return formatted_results

# 3. Enhanced UI Layout
```

```
with gr.Blocks(theme=gr.themes.Soft(primary_hue="indigo")) as demo:
    gr.Markdown("# Modern Movie Sentiment AI")
    gr.Markdown("### *Transformer-Based Sequence Classification (DistilBERT)*")

    with gr.Row():
        with gr.Column():
            input_msg = gr.Textbox(
                label="Review Text",
                placeholder="How was the movie?",
                lines=4
            )
            submit_btn = gr.Button("Run Analysis", variant="primary")

        with gr.Column():
            output_plot = gr.Label(label="Result", num_top_classes=2)

    submit_btn.click(fn=analyze_review, inputs=input_msg, outputs=output_plot)

    gr.Examples(
        examples=[
            "This was the best movie I've seen all year!",
            "What a total disaster and waste of money."
        ],
        inputs=input_msg
    )

if __name__ == "__main__":
    demo.launch()
```

Output:


Modern Movie Sentiment AI
Transformer-Based Sequence Classification (DistilBERT).
Review Text
This was the best movie I've seen all year!
Run Analysis
Examples
This was the best movie I've seen all year!
What a total disaster and waste of money.
Result
Positive
Positive 93%
Negative 7%
Use via API · Built with Gradio · Settings